Отчёт по лабораторной работе № 2

Модели SIR и Лотки–Вольтерры

Талебу Тенке Франк

Численное исследование моделей SIR и Лотки–Вольтерры в Julia: моделирование эпидемии, взаимодействия популяций, анализ графиков и фазовых портретов.

Содержание

1 Введение **6**
1.1 Актуальность . . . 7
1.2 Цель работы . . . 8
1.3 Задачи . . . 9

2 Теоретическая часть **10**

3 Материалы и методы **11**

4 Выполнение лабораторной работы **12**
4.1 Подготовка среды и запуск моделей . . . 13
4.2 Реализация модели SIR . . . 15
4.3 Динамика трёх групп . . . 17
4.4 Динамика числа заражённых . . . 18
4.5 Инфицированные в логарифмическом масштабе . . . 19
4.6 Динамика эпидемии в процентах . . . 20
4.7 Фазовый портрет SIR . . . 21
4.8 Эффективное репродуктивное число . . . 22
4.9 Комплексная панель графиков SIR . . . 23
4.10 Реализация модели Лотки–Вольтерры . . . 24
4.11 Динамика популяций . . . 25
4.12 Фазовый портрет системы . . . 26
4.13 Производные популяций . . . 27
4.14 Относительные темпы роста . . . 28
4.15 Спектральный анализ Фурье . . . 29
4.16 Комплексная панель графиков . . . 30

5 Результаты **31**

6 Обсуждение результатов **32**

7 Заключение **33**

Список литературы **34**

Список иллюстраций

Список таблиц

Студент: Талебу Тенке Франк

Студенческий номер: 1032224534

Лабораторная работа № 2: Модели SIR и Лотки–Вольтерры

1 Введение

Работа посвящена математическому моделированию динамических процессов: распространению инфекции в популяции и взаимодействию хищников и жертв.

1.1 Актуальность

Модели на основе дифференциальных уравнений позволяют исследовать развитие эпидемий и изменения численности взаимодействующих популяций во времени.

1.2 Цель работы

Изучить модели SIR и Лотки–Вольтерры, выполнить их численное моделирование в Julia и проанализировать результаты с помощью графиков.

1.3 Задачи

1. Подготовить рабочий каталог и установить необходимые пакеты Julia.
2. Запустить программы `sir_ode.jl` и `lv_ode.jl`.
3. Построить и проанализировать графики модели SIR.
4. Построить и проанализировать графики модели Лотки–Вольтерры.

2 Теоретическая часть

Модель SIR разделяет популяцию на восприимчивых S, инфицированных I и выздоровевших R. Для замкнутой популяции используется система:

$$\frac{dS}{dt} = -\beta c \frac{SI}{N}, \qquad \frac{dI}{dt} = \beta c \frac{SI}{N} - \gamma I, \qquad \frac{dR}{dt} = \gamma I.$$

Базовое репродуктивное число: $R_0 = \frac{\beta c}{\gamma}$.

Модель Лотки–Вольтерры описывает взаимодействие популяций жертв x и хищников y:

$$\frac{dx}{dt} = \alpha x - \beta xy, \qquad \frac{dy}{dt} = \delta xy - \gamma y.$$

Параметры определяют рост жертв, интенсивность поедания, эффективность преобразования пищи и смертность хищников.

3 Материалы и методы

Использованы язык Julia, численное решение обыкновенных дифференциальных уравнений и построение графиков. Основные программы — `sir_ode.jl` и `lv_ode.jl`. Результаты представлены на снимках экрана.

4 Выполнение лабораторной работы

Выполнение разделено на подготовку среды, реализацию модели SIR и реализацию модели Лотки–Вольтерры.

4.1 Подготовка среды и запуск моделей

Проверена установка пакетов Julia, запущены программы `sir_ode.jl` и `lv_ode.jl`.

```
julia> using Pkg

julia> Pkg.status()
Status `~/.julia/environments/v1.10/Project.toml`
  [6e4b80f9] BenchmarkTools v1.6.3
  [336ed68f] CSV v0.10.16
  [a93c6f00] DataFrames v1.8.1
  [0c46a032] DifferentialEquations v7.17.0
  [634d3b9d] DrWatson v2.19.1
  [7a1cc6ca] FFTW v1.10.0
^ [7073ff75] IJulia v1.34.3
  [033835bb] JLD2 v0.6.3
  [b964fa9f] LaTeXStrings v1.4.0
  [98b081ad] Literate v2.21.0
  [1dea7af3] OrdinaryDiffEq v6.108.0
  [91a5bcdd] Plots v1.41.6
  [d7167be5] Quarto v1.0.0
  [05bca326] SimpleDiffEq v1.13.0
  [f3b207a7] StatsPlots v0.15.8
Info Packages marked with ^ have new versions available and may be upgradable.

julia>
```

```
franktaleu @fedora:~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/la
b02$ julia scripts/sir_ode.jl
  Activating project at `~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/l
abs/lab02`
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 = 0.0
Could not create decoration from factory! Running with no decorations.

Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
 - Порог коллективного иммунитета: 50.0%
 - Теоретический пик при S/N = 1/R0 = 0.5
```

```
============================================================
Модель Лотки-Вольтерры (хищник-жертва)
============================================================

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0

Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени

============================================================
Анализ результатов
============================================================

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
Could not create decoration from factory! Running with no decorations.
```

4.2 Реализация модели SIR

Подключены необходимые пакеты, определена функция модели, заданы начальные условия и подготовлен запуск численного решения.

```
============================================================
Модель Лотки-Вольтерры (хищник-жертва)
============================================================

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0

Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени

============================================================
Анализ результатов
============================================================

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
Could not create decoration from factory! Running with no decorations.
```

```julia
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end
    nothing
end
```

```
δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0] # S, I, R
p = [0.05, 10.0, 0.25] # β, c, γ

# Расчет базового репродуктивного числа
R0 = (p[2] * p[1]) / p[3] # R0 = (c * β) / γ

# Создание и решение задачи
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = δt)

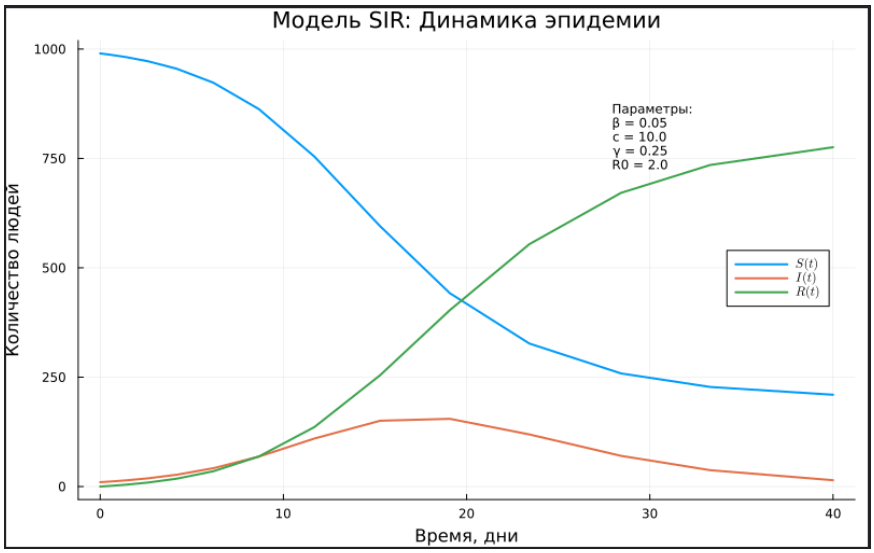
# Подготовка данных в DataFrame
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R # Общая численность популяции
```

4.3 Динамика трёх групп

Построен основной график изменения численности восприимчивых, инфицированных и выздоровевших.

```
plt1 = @df df_ode plot(:t,
     [:S :I :R],
     label=[L"S(t)" L"I(t)" L"R(t)"],
     xlabel="Время, дни",
     ylabel="Количество людей",
     title="Модель SIR: Динамика эпидемии",
     linewidth=2,
     legend=:right,
     grid=true,
     size=(800, 500))

# Добавление аннотаций с параметрами
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("Параметры:\nβ = $(p[1])\nc = $(p[2])\nγ = $(p[3])\nR0 = $(round(R0, digits=2))",
    8, :left))
```

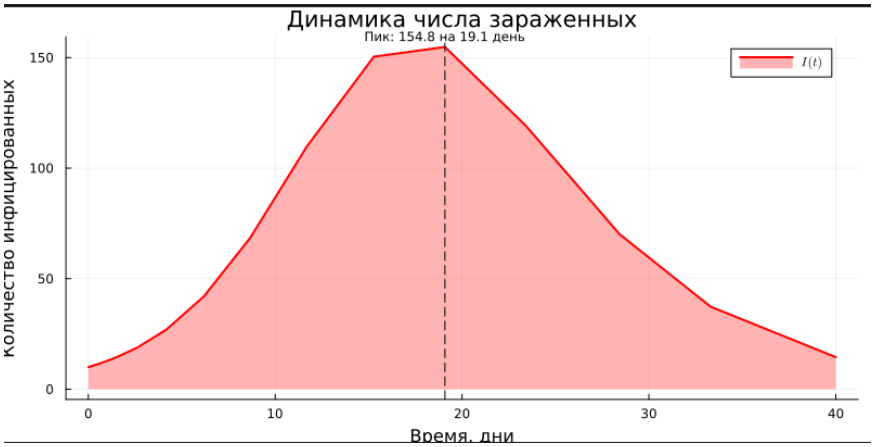


4.4 Динамика числа заражённых

Исследовано изменение численности инфицированных во времени.

```
plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество инфицированных",
    title="Динамика числа зараженных",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))

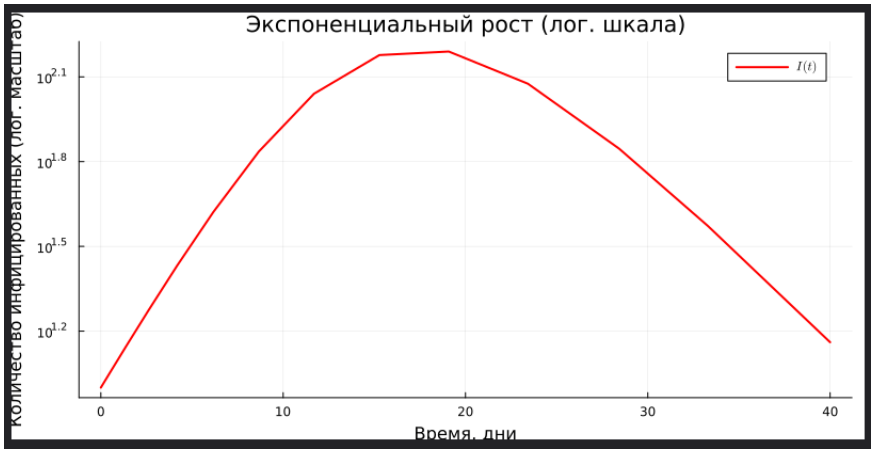
# Отметка пика эпидемии
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false, linewidth=1)
annotate!(plt2, peak_time, peak_value * 1.05,
    text("Пик: $(round(peak_value, digits=1)) на $(round(peak_time, digits=1)) день",
    8, :top))
```



4.5 Инфицированные в логарифмическом масштабе

Построена визуализация динамики инфицированных в логарифмическом масштабе.

```
# График в логарифмическом масштабе (для анализа экспоненциального роста)
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество инфицированных (лог. масштаб)",
    title="Экспоненциальный рост (лог. шкала)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))
```

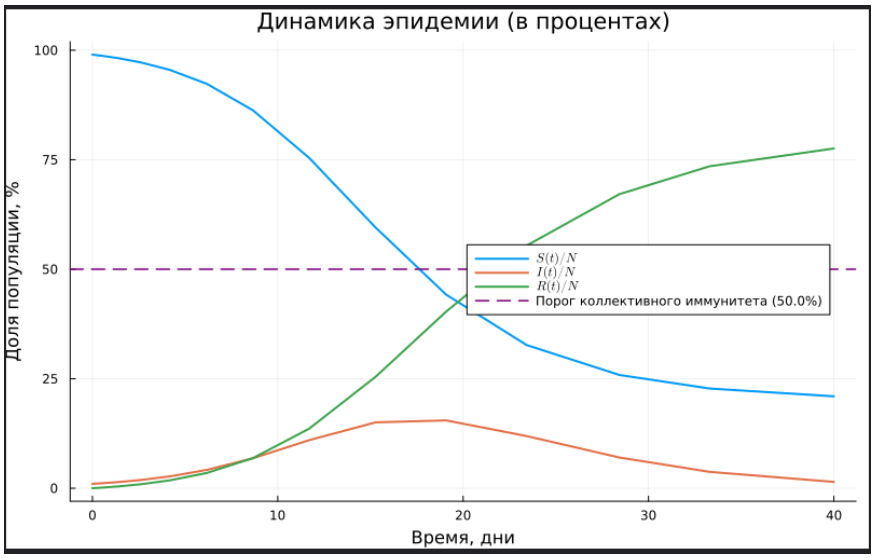


4.6 Динамика эпидемии в процентах

Построены относительные показатели эпидемического процесса.

```
plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="Время, дни",
    ylabel="Доля популяции, %",
    title="Динамика эпидемии (в процентах)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

# Горизонтальная линия для порога коллективного иммунитета
if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple, linestyle=:dash,
    label="Порог коллективного иммунитета ($(round(herd_immunity_threshold, digits=1))%)",
    linewidth=1.5)
end
```

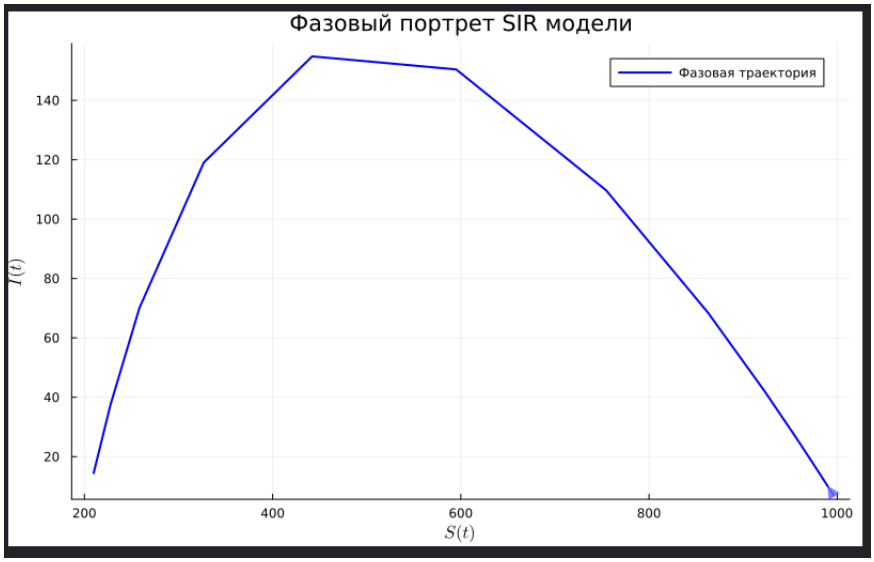


4.7 Фазовый портрет SIR

Построен фазовый портрет модели SIR.

```
plt5 = plot(df_ode.S, df_ode.I,
    label="Фазовая траектория",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="Фазовый портрет SIR модели",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    legend=:topright)

# Добавление стрелок направления
for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.I[i+1]],
          arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```



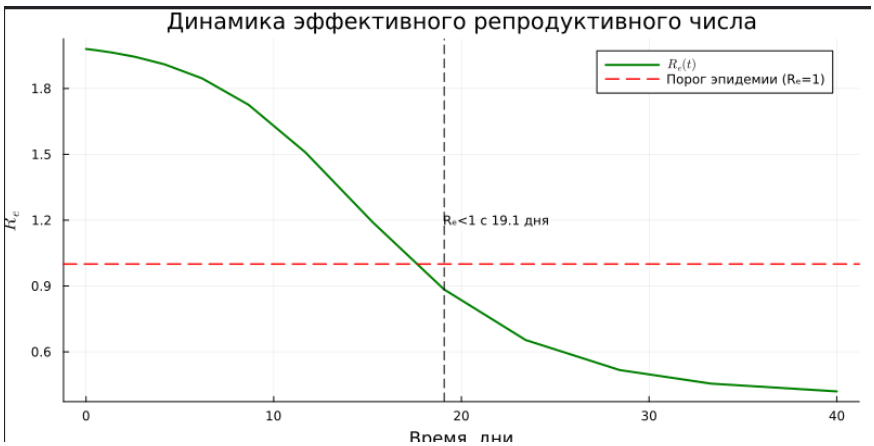
4.8 Эффективное репродуктивное число

Исследовано изменение эффективного репродуктивного числа.

```
# График Rₑ - эффективного репродуктивного числа
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="Время, дни",
    ylabel=L"R_e",
    title="Динамика эффективного репродуктивного числа",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))

# Горизонтальная линия на уровне 1
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="Порог эпидемии (Rₑ=1)", linewidth=1.5)

# Отметка момента, когда Rₑ становится < 1
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=false, linewidth=1)
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 с $(round(cross_time, digits=1)) дня", 8, :left))
end
```



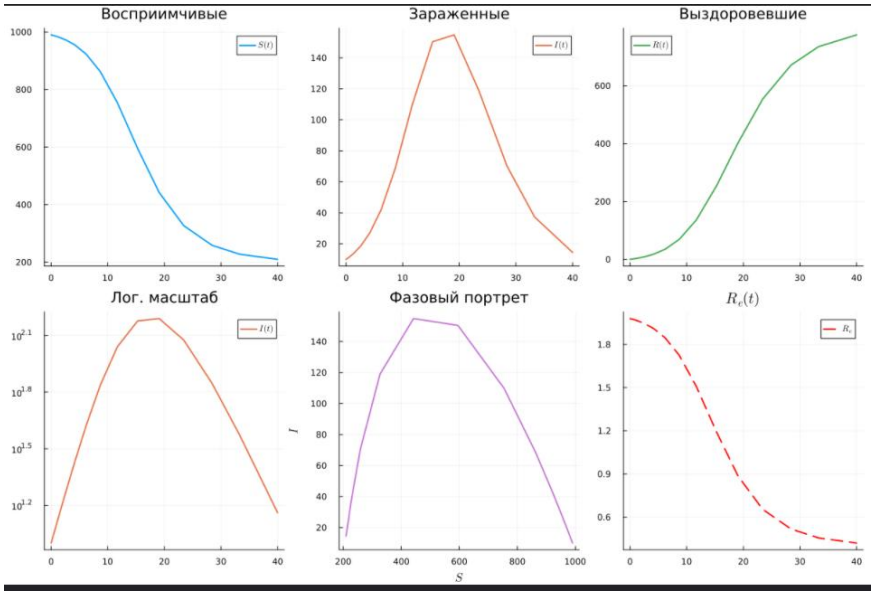
4.9 Комплексная панель графиков SIR

Сформирована сводная панель визуализаций модели SIR.

```
# Компактный график всех кривых в одной панели
plt7 = plot(layout=(2, 3), size=(1200, 800))

# Верхний ряд
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidth=2, title="Восприимчивые")
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2, title="Зараженные")
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidth=2, title="Выздоровевшие")

# Нижний ряд
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2,
    yscale=:log10, title="Лог. масштаб")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=2,
    title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, linewidth=2,
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```



4.10 Реализация модели Лотки–Вольтерры

Подключены пакеты, определена функция модели, заданы начальные условия и выполнено решение задачи.

```
using DrWatson
@quickactivate

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

function lotka_volterra!(du, u, p, t)
    x, y = u # x - жертвы, y - хищники
    α, β, δ, γ = p # параметры модели
    @inbounds begin
    du[1] = α*x - β*x*y # уравнение для жертв
    du[2] = δ*x*y - γ*y # уравнение для хищников
    end
    nothing
end

# Начальные условия: [жертвы, хищники]
u0_lv = [40.0, 9.0] # начальная популяция

# Временные параметры
tspan_lv = (0.0, 200.0) # длительность симуляции
dt_lv = 0.01 # шаг интегрирования

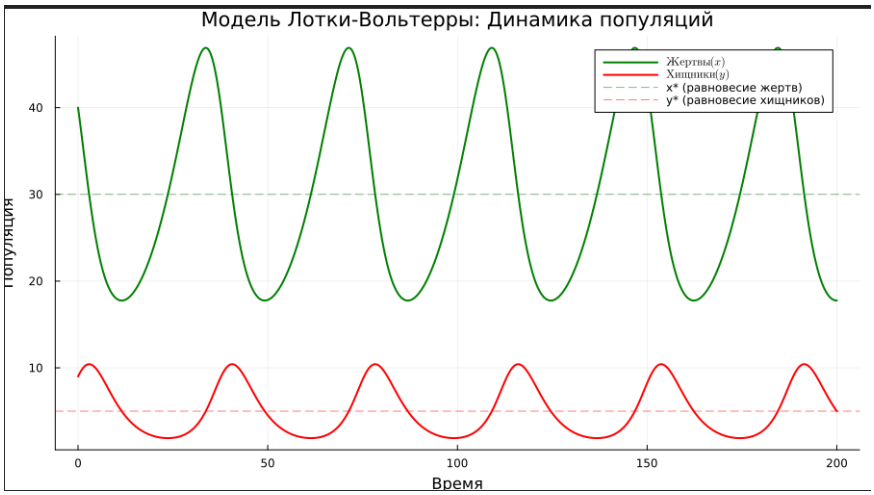
# Создание и решение задачи
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
               dt = dt_lv,
               Tsit5(), # Метод 5-го порядка
               reltol=1e-8, # Относительная точность
               abstol=1e-10, # Абсолютная точность
               saveat=0.1, # Сохраняем каждые 0.1 единицы времени
               dense=true # Включаем плотный вывод для интерполяции
)
```

4.11 Динамика популяций

Построены зависимости численности жертв и хищников от времени.

```
# График 1: Динамика популяций во времени
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"Жертвы (x)" L"Хищники (y)"],
    xlabel="Время",
    ylabel="Популяция",
    title="Модель Лотки-Вольтерры: Динамика популяций",
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500),
    color=[:green :red])

# Добавление стационарных уровней
# Добавление стационарных уровней
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, label="x* (равновесие жертв)")
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, label="y* (равновесие хищников)")
```



4.12 Фазовый портрет системы

Построена фазовая траектория взаимодействующих популяций.

```
# График 2: Фазовый портрет (хищники vs жертвы)
plt2 = plot(df_lv.prey, df_lv.predator,
    label="Фазовая траектория",
    xlabel="Популяция жертв (x)",
    ylabel="Популяция хищников (y)",
    title="Фазовый портрет системы",
    color=:blue,
    linewidth=1.5,
    grid=true,
    size=(800, 600),
    legend=:topright)

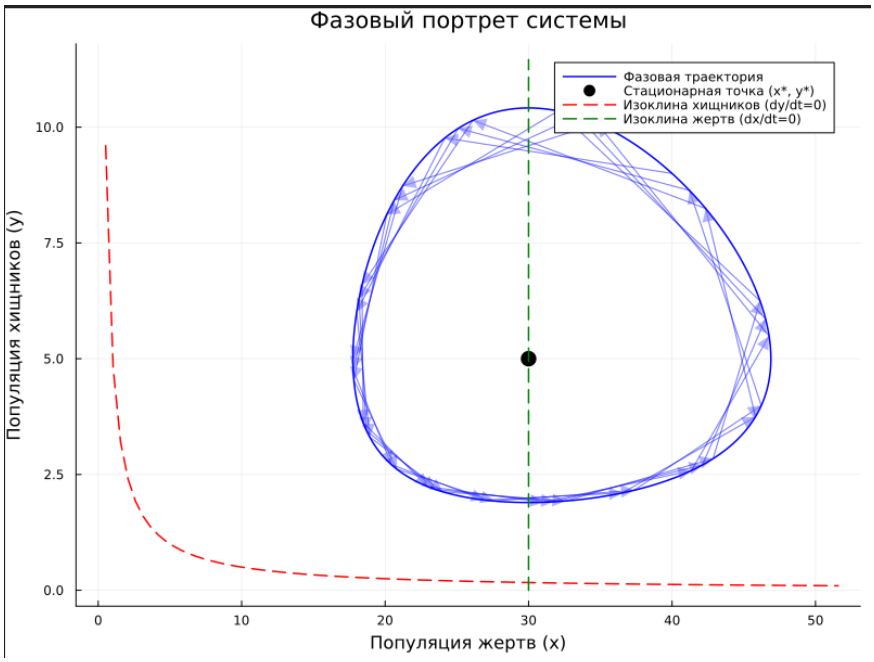
# Добавление стрелок направления на фазовом портрете

step = 50 # шаг для отображения стрелок
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
      [df_lv.predator[i], df_lv.predator[i+step]],
      arrow=:closed, color=:blue, alpha=0.3, label=false)
end

# Добавление стационарной точки
scatter!(plt2, [x_star], [y_star],
    color=:black, markersize=8, label="Стационарная точка (x*, y*)")

# Изоклины (нулевого роста)
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-изоклина (dy/dt=0)
plot!(plt2, x_range, y_nullcline,
    color=:red, linestyle=:dash, linewidth=1.5, label="Изоклина хищников (dy/dt=0)")

y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) # x-изоклина (dx/dt=0)
plot!(plt2, x_nullcline, y_range,
    color=:green, linestyle=:dash, linewidth=1.5, label="Изоклина жертв (dx/dt=0)")
```

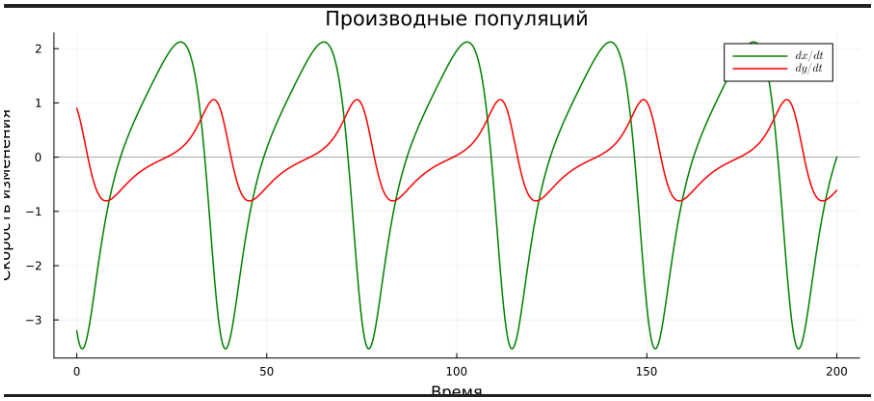


4.13 Производные популяций

Исследованы скорости изменения численности двух популяций.

```
# График 3: Производные (скорости изменения)
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
    xlabel="Время",
    ylabel="Скорость изменения",
    title="Производные популяций",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])

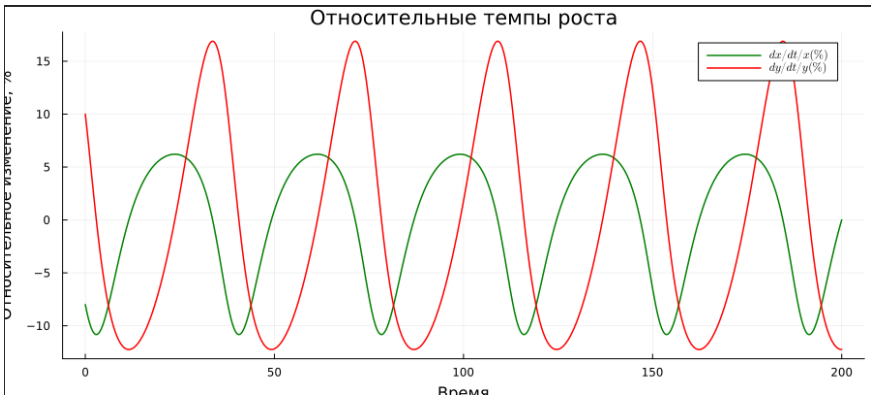
hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=false)
```



4.14 Относительные темпы роста

Построены относительные темпы роста популяций.

```
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .* 100
plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="Время",
    ylabel="Относительное изменение, %",
    title="Относительные темпы роста",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])
```



4.15 Спектральный анализ Фурье

Выполнен спектральный анализ временных рядов модели.

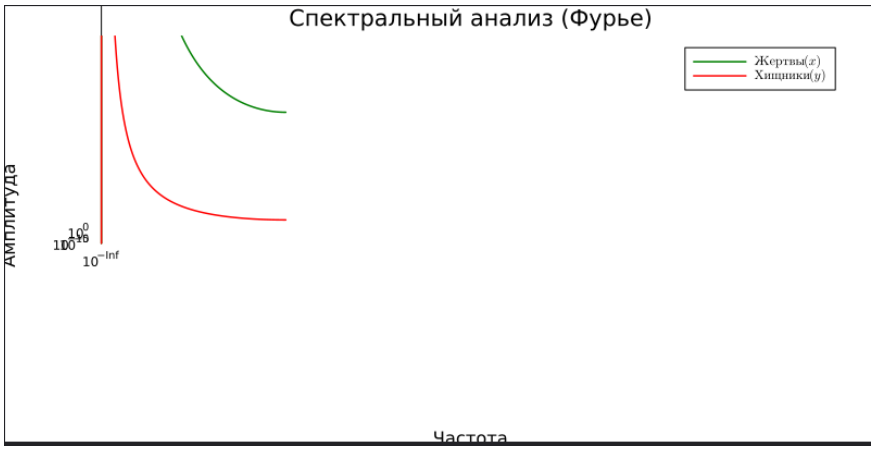
```
function compute_fft(signal, dt)
    n = length(signal)
    # Используем rfft для вещественных сигналов (возвращает только положительные частоты)
    spectrum = abs.(rfft(signal))
    # Соответствующие частоты для rfft
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
end

# Вычисление периодов колебаний
freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey), dt_lv)
freq_predator, spectrum_predator = compute_fft(df_lv.predator .- mean(df_lv.predator), dt_lv)

plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"Жертвы (x)" L"Хищники (y)"],
    xlabel="Частота",
    ylabel="Амплитуда",
    title="Спектральный анализ (Фурье)",
    linewidth=1.5,
    xscale=:log10,
    yscale=:log10,
    legend=:topright,
    grid=true,
    size=(800, 400),
    color=[:green :red])

# Нахождение доминирующих частот
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1 # пропускаем нулевую частоту
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1/dominant_freq_prey
    println("\nДоминирующая частота колебаний жертв: ", round(dominant_freq_prey, digits=4), " Гц")
    println("Период колебаний жертв: ", round(period_prey, digits=2), " единиц времени")

end
```

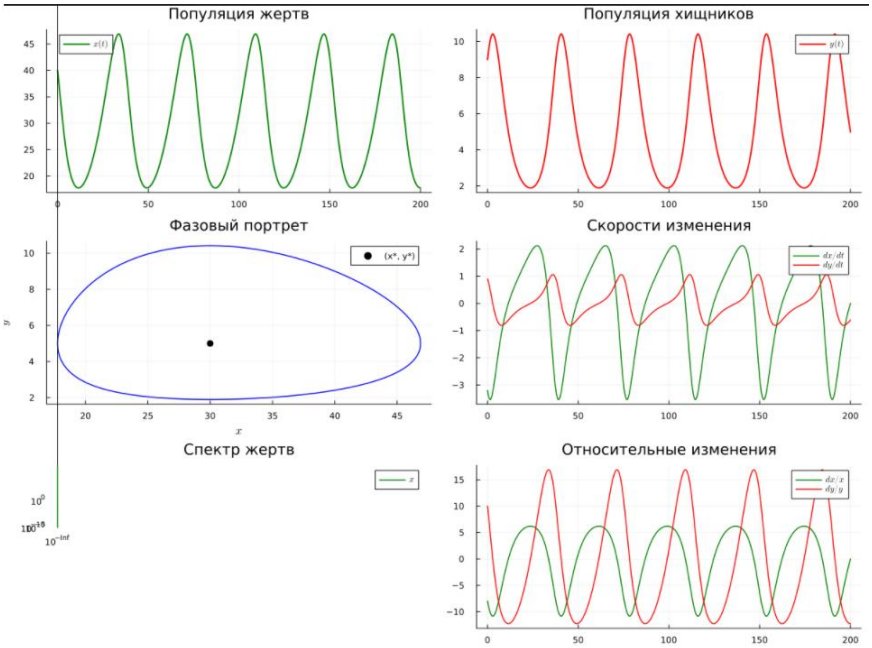


4.16 Комплексная панель графиков

Сформирована сводная визуализация результатов моделирования.

```
# График 6: Компактная панель всех графиков
plt6 = plot(layout=(3, 2), size=(1200, 900))

plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, linewidth=2,
      title="Популяция жертв", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red, linewidth=2,
      title="Популяция хищников", grid=true)
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue, linewidth=1.5,
      title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, label="(x*, y*)")
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
      label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
      title="Скорости изменения", grid=true, legend=:topright)
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green, linewidth=1.5,
      title="Спектр жертв", xscale=:log10, yscale=:log10, grid=true)
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
      label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
      title="Относительные изменения", grid=true, legend=:topright)
```



5 Результаты

Запущены модели SIR и Лотки–Вольтерры. Для SIR получены графики динамики групп, инфицирования, фазовый портрет и оценка эффективного репродуктивного числа. Для модели Лотки–Вольтерры получены временные зависимости, фазовый портрет, производные, относительные темпы роста, спектральный анализ и сводные графики.

6 Обсуждение результатов

Результаты модели SIR позволяют проследить изменение численности восприимчивых, инфицированных и выздоровевших. В модели Лотки–Вольтерры графики иллюстрируют колебательную динамику численности жертв и хищников. Фазовые портреты показывают взаимосвязь переменных.

7 Заключение

Выполнена лабораторная работа № 2. Проведено моделирование двух систем дифференциальных уравнений в Julia и подготовлены графики для анализа динамики эпидемии и взаимодействия популяций.

Список литературы

1. Материалы лабораторной работы № 2 «Модели SIR и Лотки–Вольтерры» (предоставленный отчёт).